

Sprint 4 – Podsumowanie prac

Nazwa kodowa projektu: SwimSync

Projekt: System organizacji szkoły pływania

Zespół

- Wojciech Osowski (Lider Zespołu)
- Emilia Wojtowicz
- Mateusz Strózek

Linki

- Repozytorium: [link](#)
- Tablica projektowa: [link](#)

1. Cele i założenia etapu

Głównym celem ostatniego sprintu była fizyczna implementacja zaprojektowanej wcześniej architektury backendowej, pokrycie jej zautomatyzowanymi testami jednostkowymi oraz integracyjnymi, a także sfinalizowanie dokumentacji i przygotowanie prezentacji końcowej. Etap ten wieńczy prace nad projektem i dostarcza w pełni funkcjonalny, przetestowany prototyp logiki biznesowej (backend).

2. Zrealizowane zadania i szczegółowe podsumowanie prac

A. Implementacja projektu (Wojciech Osowski)

Zgodnie z zatwierdzonymi diagramami UML wdrożono kompletną logikę backendową w języku Python. Zaimplementowano 4 warstwy architektoniczne (wzorzec MVC):

- **Warstwa Encji (`entities.py`):** Zaimplementowano klasy domenowe (`ProfilUzytkownika`, `TorBasenowy`, `GrupaPlywacka`, `Rezerwacja`, `Obecnosc`). Dodano do nich metody serializacji/deserializacji (`to_dict`, `from_dict`) oraz logikę biznesową (np. algorytm sprawdzania wolnych miejsc uwzględniający jednocześnie limit grupy i bezwzględny limit toru).
- **Warstwa Bazy Danych (`database.py`):** Ponieważ zrezygnowano z zewnętrznych silników SQL, stworzono autorski, plikowy system zapisu oparty na formacie JSON. Zaimplementowano automatyczne generowanie katalogu `data/`, zarządzanie operacjami wejścia/wyjścia, masowy zapis (`bulk_insert`) dla optymalizacji list obecności oraz wyszukiwanie konfliktów dat (parsowanie formatu ISO).

- **Warstwa Kontrolerów (`controllers.py`):** Zaimplementowano kluczową logikę sterującą:
 - Wdrożono blokadę *overbookingu* – system dynamicznie odrzuca próby zapisu po przekroczeniu limitu.
 - Zaimplementowano mechanizm *Soft Delete* dla anulowania rezerwacji (zamiana statusu na `anulowana`) z rygorystyczną weryfikacją wyprzedzenia czasowego (minimum 24 godziny).
 - Zaimplementowano mechanizm weryfikacji harmonogramu, uniemożliwiający administratorowi tworzenie konfliktów na torach lub przypisywanie instruktora do dwóch grup jednocześnie.
- **Warstwa Prezentacji (`boundaries.py`):** Stworzono symulację interfejsów (m.in. `PortalZapisow`, `InterfejsWebowy`), które przechwytyją błędy z kontrolerów i wyprowadzają sformatowane komunikaty na standardowe wyjście konsoli.
- **Izolacja środowiska:** Napisano specjalne *fixtures* (`temp_db`, `controllers`), wykorzystujące wbudowany mechanizm `tmp_path`. Dzięki temu każdy test tworzy własną, tymczasową bazę JSON, nie uszkadzając właściwych danych aplikacji.
- **Testowanie Encji i DB:** Zweryfikowano poprawne generowanie haseł, działanie algorytmów zliczających frekwencję oraz spójność operacji zapis/odczyt plików JSON.
- **Testowanie Logiki (Edge Cases):** Przetestowano zachowanie systemu w skrajnych przypadkach: wymuszanie rzucania wyjątków `ValueError` przy próbie zapisu do przepełnionej grupy, próbie anulowania lekcji zbyt późno, czy podczas próby zmniejszenia fizycznej pojemności toru poniżej aktualnej liczby zapisanych kursantów.
- **Testowanie Interfejsu:** Za pomocą mechanizmu `capsys` przetestowano prawidłowość komunikatów zwracanych użytkownikowi.

B. Przygotowanie prezentacji (Mateusz Strózek)

Stworzono profesjonalną prezentację (pitch deck) podsumowującą projekt. Slajdy obejmują zdefiniowany problem i wizję systemu, przegląd makiety interfejsu (Hi-Fi) na podstawie badań UI/UX z poprzedniego sprintu, strukturę wdrożonej architektury oraz demonstrację możliwości przetestowanego backendu.

C. Skompletowanie dokumentacji (Emilia Wojtowicz)

Zebrano wszystkie artefakty (wizję, epiki, historyjki, raporty z badań UI, diagramy UML, dokumentację kodu i testów) w jeden, ustrukturyzowany dokument końcowy.